

# Aula 5 (Módulo 01) — Logs: syslog e journalctl

## 1. O que é um log (visão simples)

Um **log** é um registro de "o que aconteceu, quando aconteceu". Todo sistema, serviço e aplicação relevante gera logs — é a memória escrita do que o sistema fez. Sem log, uma investigação de segurança é basicamente impossível: você não consegue provar (nem reconstruir) nada que não foi registrado em algum lugar.

Já usamos logs sem formalizar isso: `/var/log/auth.log` (Aula 1 e Aula 4 deste módulo) já é um log.

## 2. Duas formas de acessar logs no Linux hoje

### syslog — o modelo clássico, baseado em arquivos texto

Historicamente, serviços Linux enviam mensagens para um "daemon de syslog" (como `rsyslog` ou `syslog-ng`), que decide em qual arquivo de `/var/log/` gravar cada mensagem, com base em categoria (chamada de **facility**: `auth`, `mail`, `cron`, `kern` ...) e **severidade** (`emerg`, `alert`, `crit`, `err`, `warning`, `notice`, `info`, `debug`).

Arquivos comuns em `/var/log/`:

| Arquivo                                                            | Conteúdo                       |
|--------------------------------------------------------------------|--------------------------------|
| <code>auth.log</code> (Debian/Ubuntu) / <code>secure</code> (RHEL) | autenticação: login, sudo, SSH |
| <code>syslog</code> (Debian/Ubuntu) / <code>messages</code> (RHEL) | mensagens gerais do sistema    |
| <code>kern.log</code>                                              | mensagens do kernel            |
| <code>cron.log</code>                                              | execuções do cron              |

### journald / journalctl — o modelo moderno (systemd)

Sistemas com `systemd` (a maioria das distribuições atuais) também têm o **journald**, que coleta logs em um formato **binário estruturado** (não é texto puro) e é consultado com o comando `journalctl`. Muitas distribuições modernas usam os dois em conjunto: `journald` como coletor central, e regras que também espelham parte disso para os arquivos de `/var/log/` no formato clássico.

```
journalctl                # todos os logs, do mais antigo ao mais recente
journalctl -f             # "follow" – acompanha em tempo real (como tail -f)
journalctl -u ssh         # logs só do serviço ssh
journalctl --since "1 hour ago" # filtra por tempo
journalctl -p err         # filtra por severidade (err ou mais grave)
journalctl -k             # só mensagens do kernel
```

**Por que importa para segurança:** `journalctl` é hoje o ponto de partida mais comum para investigar um host Linux individual — ele já vem com filtro por serviço, tempo e severidade embutido, sem precisar combinar vários comandos como `grep` / `cat` em arquivos texto separados.

### 3. Formato de uma linha de log (syslog clássico)

```
Set 19 14:32:07 servidor sshd[1234]: Failed password for root from 203.0.113.45 port 41522 ssh2
```

|            |            |            |            |

|            |            |            |            |            ↳ PID do processo            ↳ mensagem (o que aconteceu)

|            |            |            ↳ nome do processo/serviço que gerou o log

|            |            ↳ hostname da máquina

↳ timestamp

Repare: **timestamp**, **hostname**, **processo/PID** e **mensagem** — os mesmos quatro elementos, em formatos diferentes, aparecem em praticamente todo log que você vai analisar daqui para frente (inclusive nos "Alertas de SOC" simulados que vamos praticar mais adiante).

### 4. `logrotate` — por que os logs não crescem para sempre

Os arquivos de log seriam gigantes em pouco tempo se nada fosse feito. O `logrotate` é o serviço responsável por, periodicamente, "girar" os logs: renomeia o arquivo atual (ex.: `auth.log` → `auth.log.1`), comprime versões antigas (`auth.log.2.gz`) e, depois de um tempo configurado, apaga as mais antigas.

**Por que importa para segurança:** isso tem uma implicação prática direta em investigação — **logs antigos podem já ter sido apagados** pelo logrotate antes de você começar a investigar um incidente. É uma das razões pelas quais ambientes profissionais enviam logs para um sistema central (SIEM — Módulo 09) que guarda cópias por muito mais tempo do que o host local guardaria sozinho.

### 5. Exercício prático

```
journalctl -u ssh --since "24 hours ago" | tail -20
journalctl -p err --since "24 hours ago" | tail -20
```

Se o primeiro comando não retornar nada, tente `journalctl -u sshd` (o nome do serviço varia um pouco entre distribuições). Observe: mesmo sem nenhum evento "interessante", uma saída vazia já é informação — confirma que não houve atividade SSH nas últimas 24h nessa máquina, o que é sua **linha de base** para comparar depois.

### 6. Recapitulando

- Log = registro do que aconteceu; base de qualquer investigação.
- `syslog` clássico grava em arquivos texto por categoria em `/var/log/`; `journald` / `journalctl` é o modelo moderno, estruturado, consultável por serviço/tempo/severidade.
- Toda linha de log carrega, de alguma forma: timestamp, host, processo/serviço, e a mensagem em si.
- `logrotate` limita quanto tempo os logs ficam disponíveis localmente — por isso ambientes profissionais centralizam logs em um SIEM.

## Próxima aula

---

Rede no Linux: sockets, conexões, portas e um primeiro contato com firewall — os comandos `ss` e `ip`, que respondem "quem está conectado a quem, agora, nesta máquina?".

## Fontes recomendadas

---

- **RFC 5424 — "The Syslog Protocol"**: padrão técnico oficial (IETF) que define facility, severidade e formato de mensagem syslog. <https://www.rfc-editor.org/rfc/rfc5424>
- **freedesktop.org — documentação oficial do `journalctl`**: referência completa de filtros e opções. <https://www.freedesktop.org/software/systemd/man/latest/journalctl.html>